



IC Lab HW5

ViT accelerator

王彦智
2025/11/06

Introduction

- Task
 - Image classification
- Dataset:
 - CIFAR-10
 - 32X32 RGB image
- Model
 - Transformer
 - *ViT*(Vison in Transformer)

airplane



automobile



bird



cat



deer



dog



frog



horse



ship



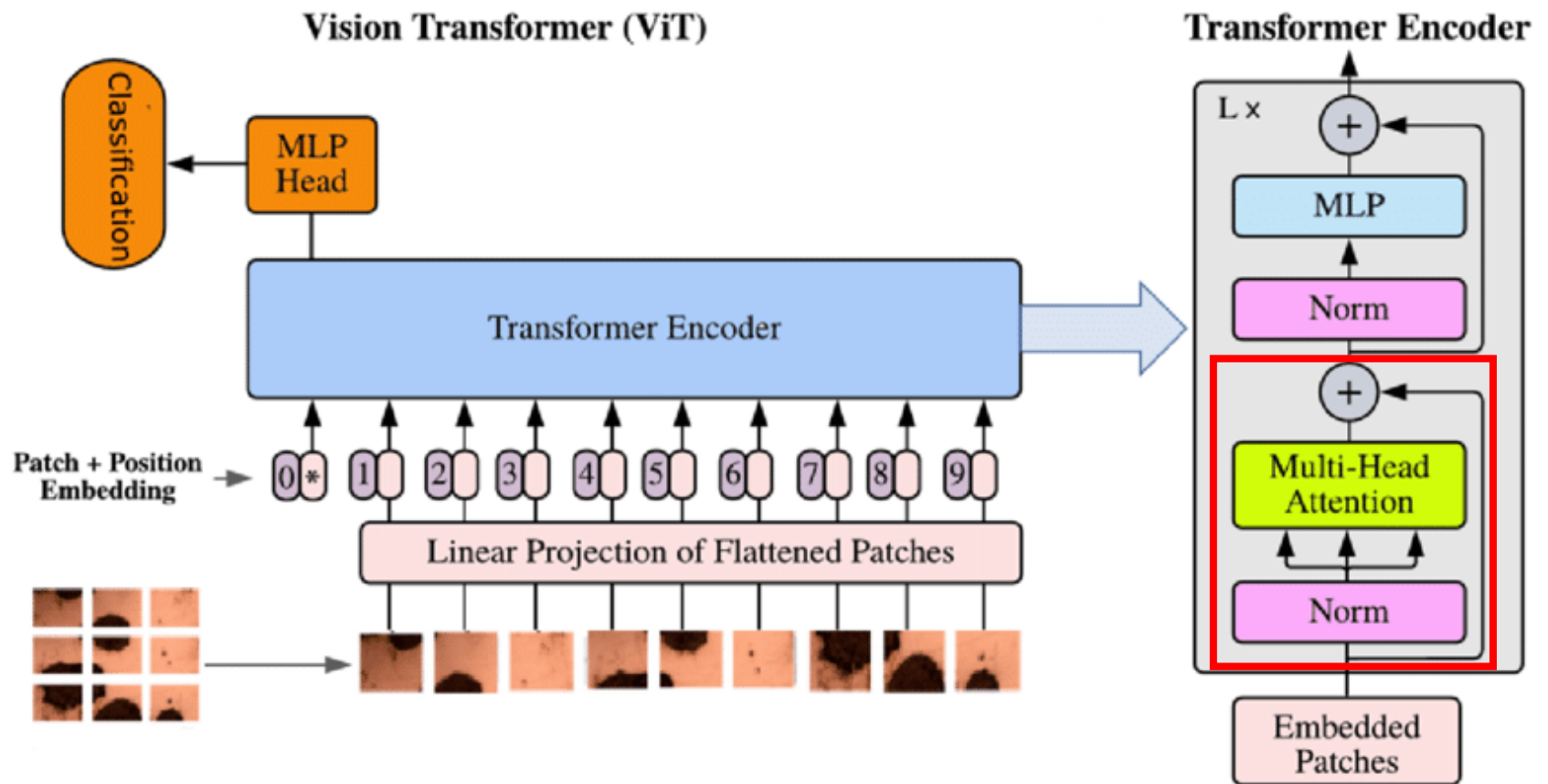
truck



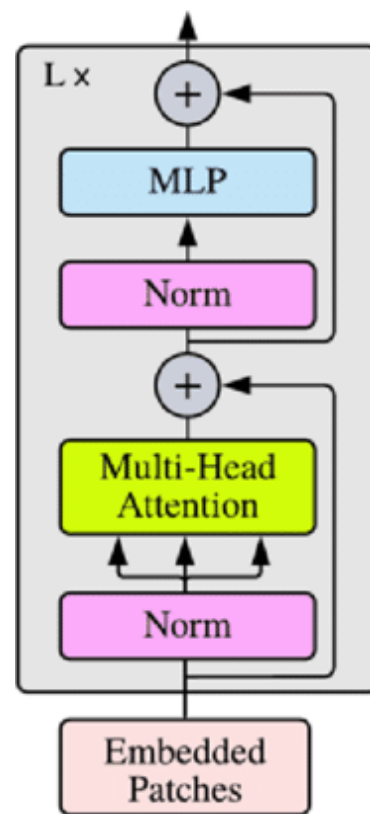
[1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, N. Houlsby, “An image is worth 16x16 words: transformer for image recognition at scale”

Model

- You need to finish Multi-Head Attention and Norm in this assignment

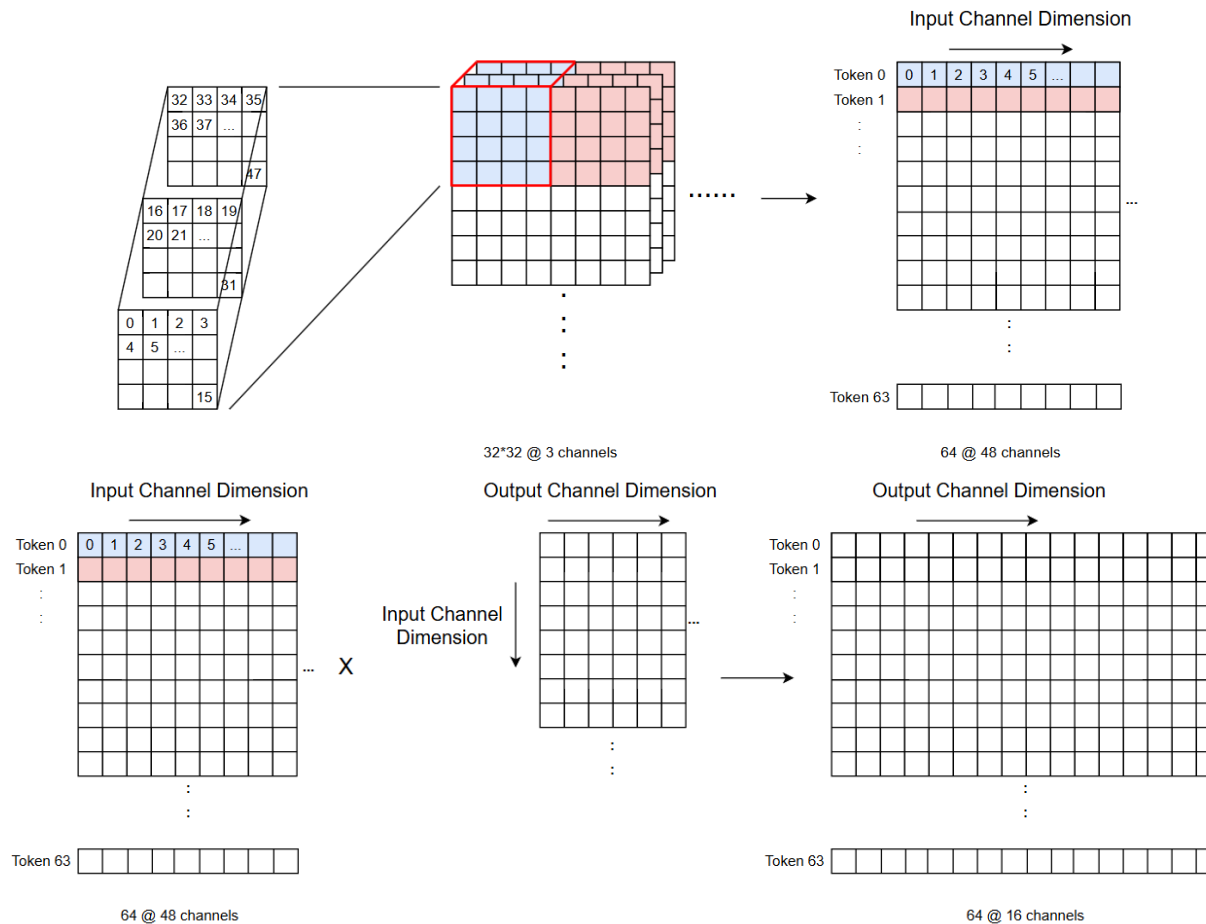


Algorithm



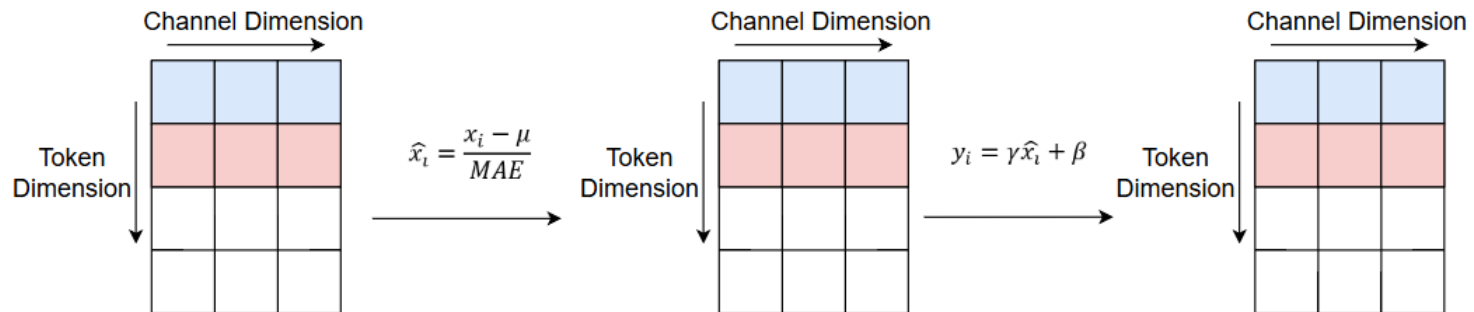
Patch + Position Embedding

- $32 * 32 \text{ pixels @ } 3 \text{ channels} \rightarrow 64 \text{ patches @ } 16 \text{ channels}$

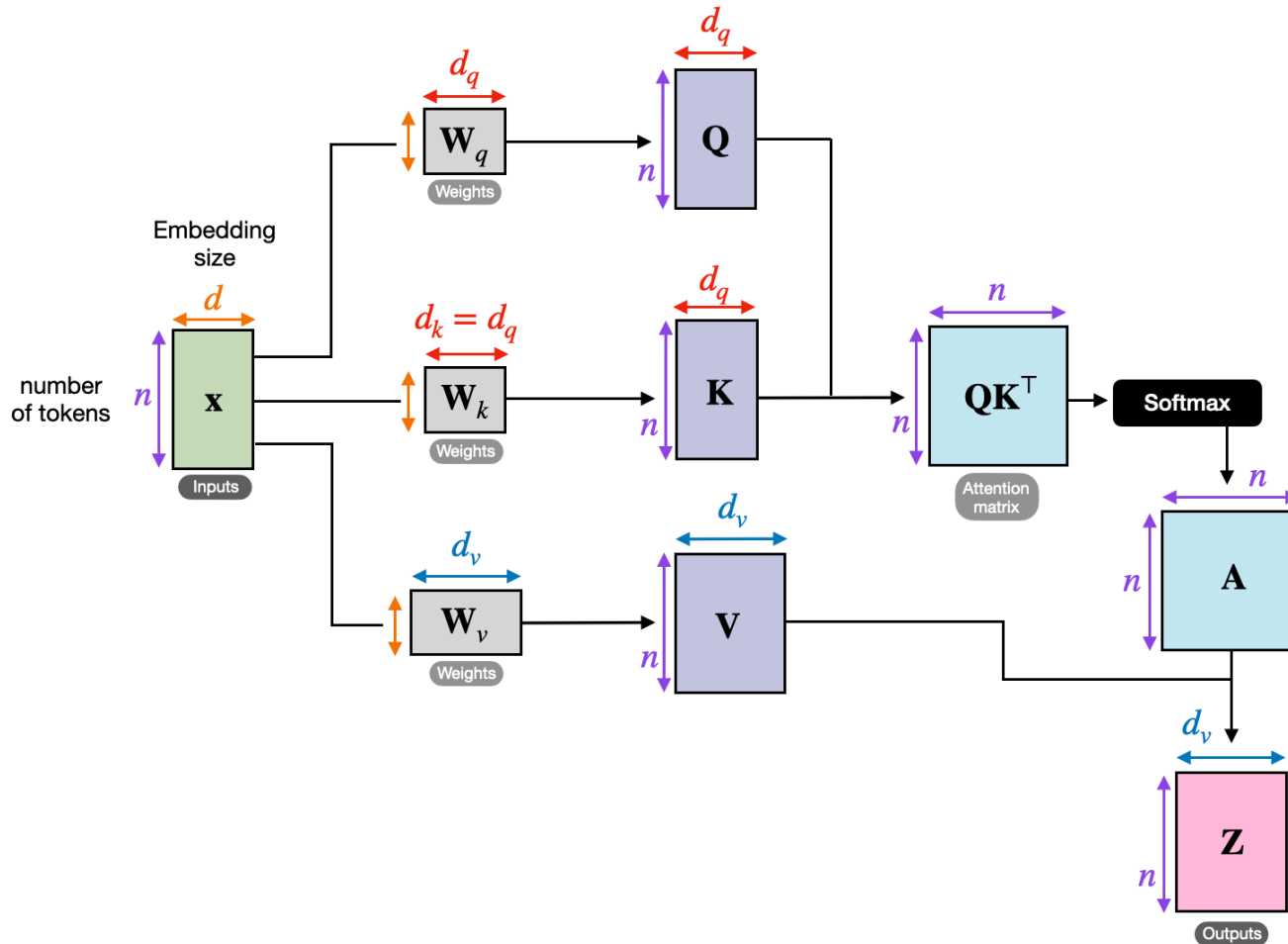


Normalization

- Standard *ViT* model applies layer normalization.
- For simplicity, we use L1 layer normalization in this assignment.
 - $\mu = \frac{1}{c} \sum_{i=0}^{c-1} x_i, MAE = \frac{1}{c} \sum_{i=0}^{c-1} |x_i - \mu|$
 - $\hat{x}_i = \frac{x_i - \mu}{MAE}$
 - $y_i = \gamma_i * \hat{x}_i + \beta_i$, where $\gamma = [\gamma_0, \gamma_1, \gamma_2, \dots, \gamma_{c-1}]$, $\beta = [\beta_0, \beta_1, \beta_2, \dots, \beta_{c-1}]$

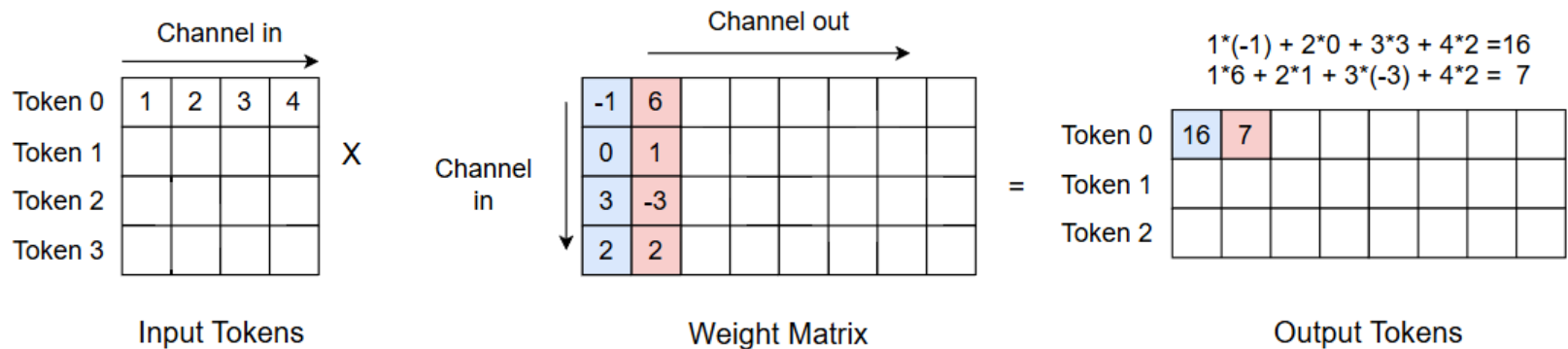


Multi-head Self-attention



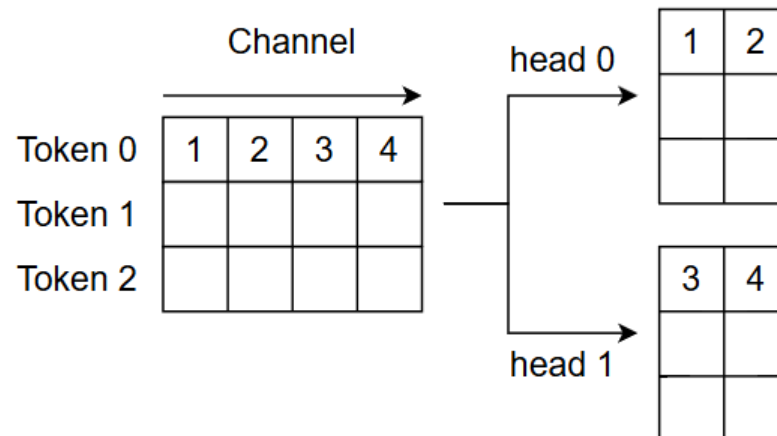
Multi-head Self-attention

- Step 1: map the input tokens into four new spaces: Q (Query), K (Key), V (Value) and O (Output, after attention)
- *64 patches @16 channels → 64 patches @16 channels*



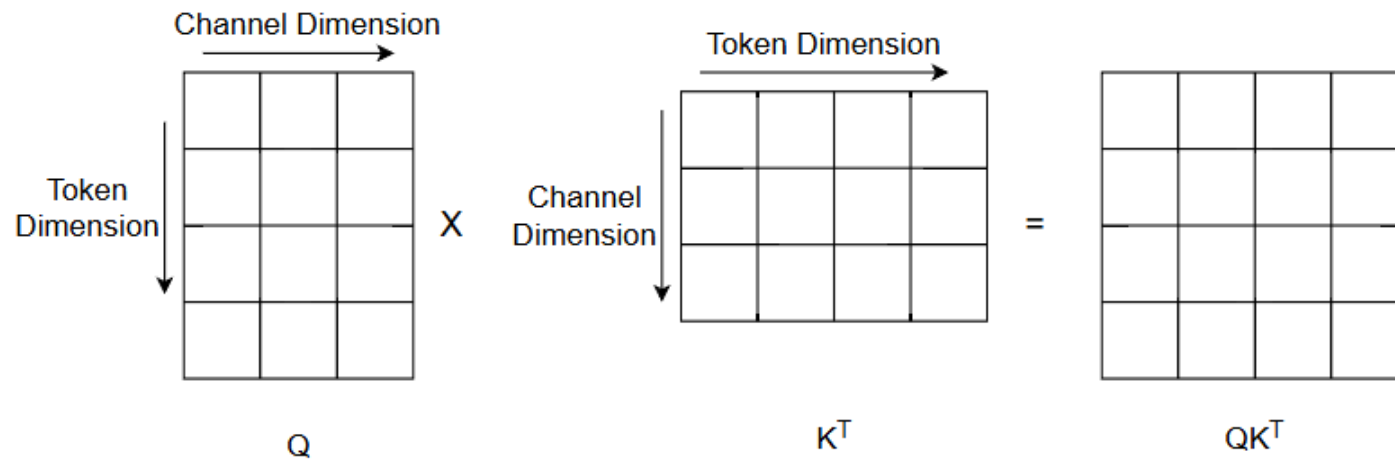
Multi-head Self-attention

- Step 2: Separate Q (Query), K (Key), V (Value) into 2 heads
- *64 patches @16 channels $\rightarrow$ 64 patches @(8 channel + 8 channels)*



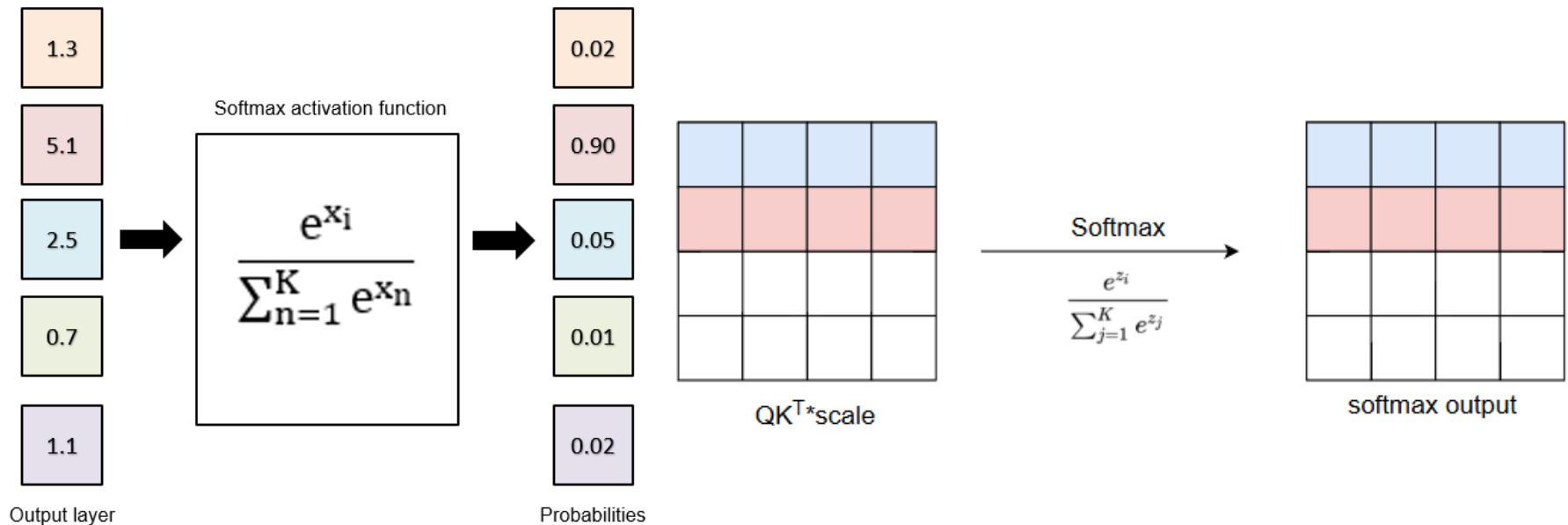
Multi-head Self-attention

- Step 3: $\frac{QK^T}{\sqrt{c_{head}}}$
- $\sqrt{c_{head}} = \sqrt{8}$
- *shape*: $Q_{64*8} @ K_{8*64}^T = (QK^T)_{64*64}$



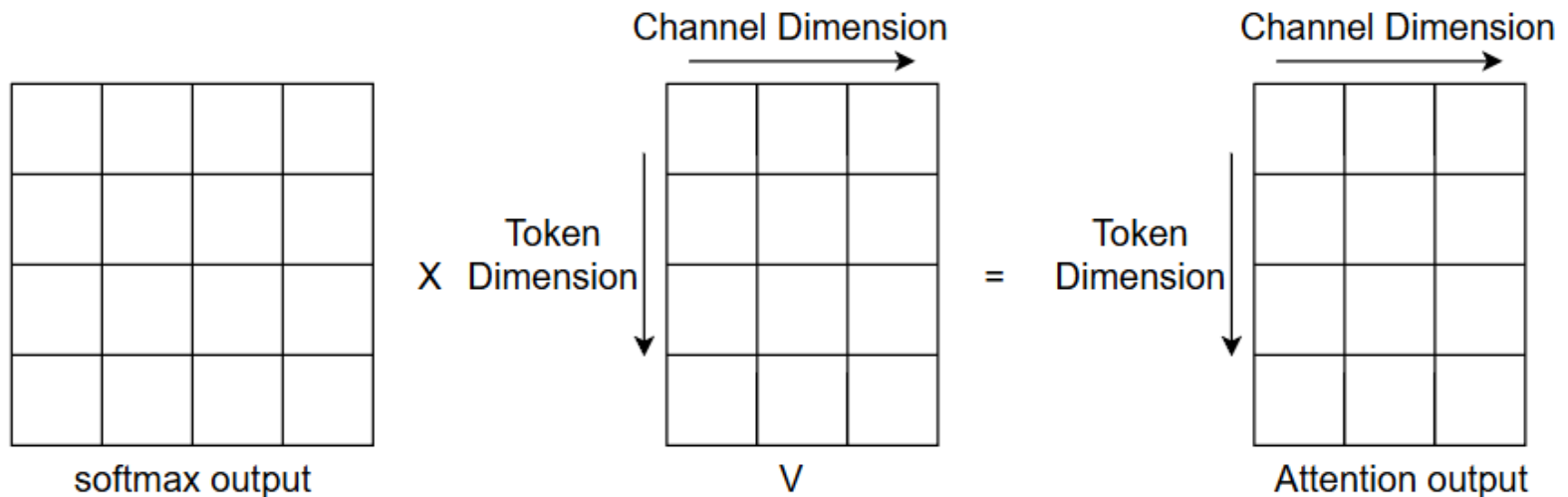
Multi-head Self-attention

- Step 4: $\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{n=1}^K e^{x_n}}$ for $i = 1, \dots, K$
- Applied to each row.



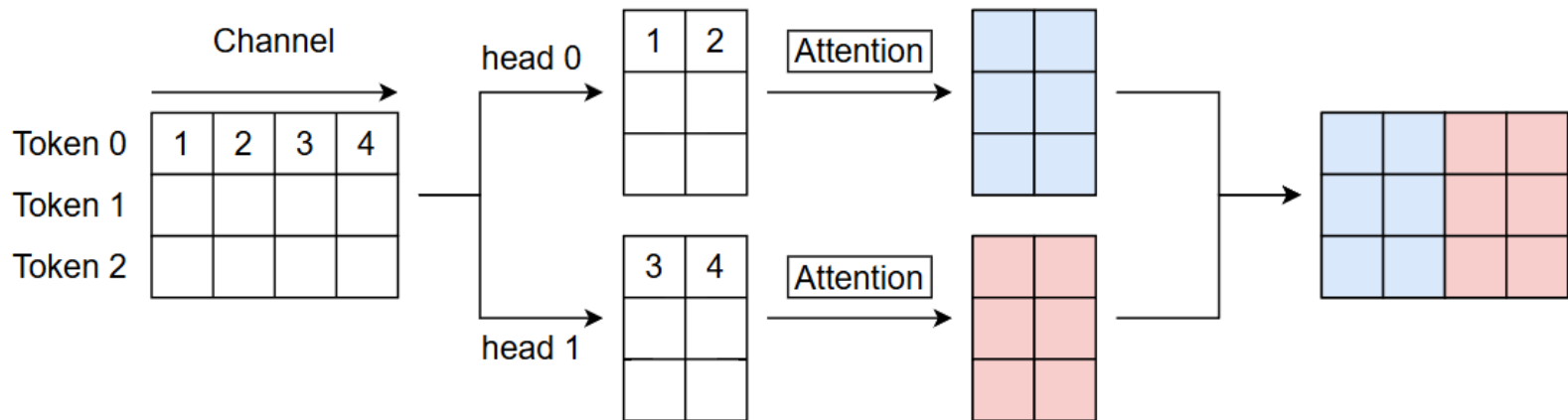
Multi-head Self-attention

- Step 5: *Multiply V*
- $\text{Softmax_out}_{64 \times 64} @ V_{64 \times 8} = \text{Attention_out}_{64 \times 8}$



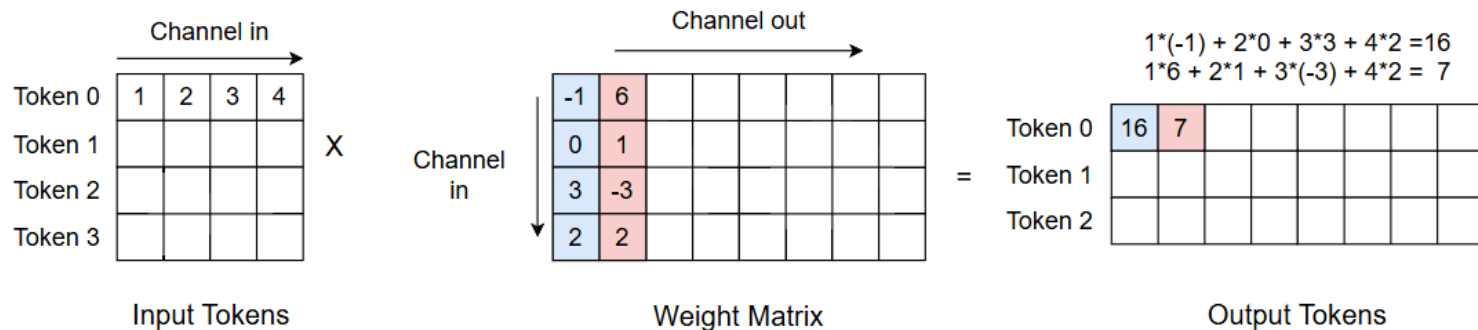
Multi-head Self-attention

- Step 6: concatenate 2 heads
- *shape: 64 patches @16 channels*



Multi-head Self-attention

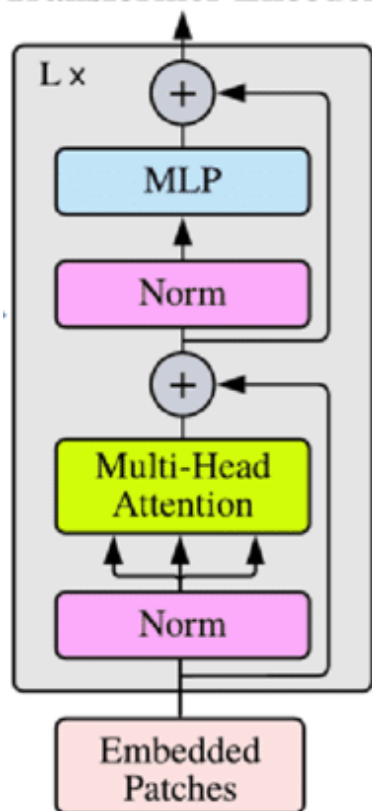
- Step 7: O(Output) mapping
- *64 patches @16 channels → 64 patches @16 channels*



Residue Add

- 64 patches @16 channels + 64 patches @16 channels

Transformer Encoder



$$\boxed{1} + \boxed{2} = \boxed{3}$$

1	1	0	2
-7	0	-4	0
0	-4	6	7
4	3	3	1

Convolve output

+

2	5	0	1
3	2	-3	-1
0	2	6	8
1	5	1	2

Residue

=

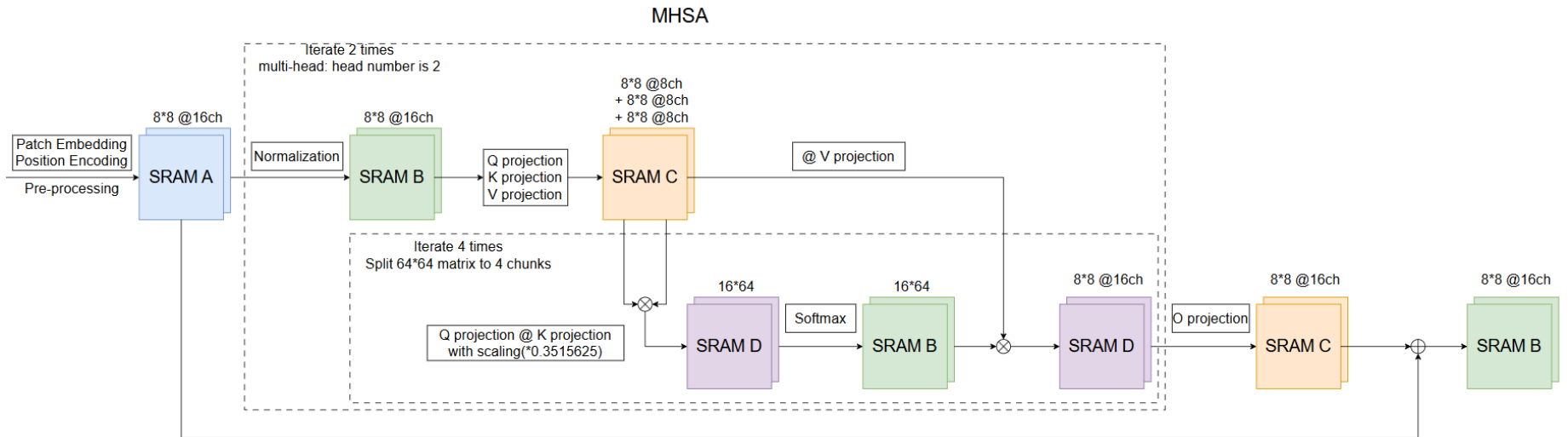
3	6	0	3
-4	2	-7	-1
0	-2	12	15
5	8	4	3

Output feature map

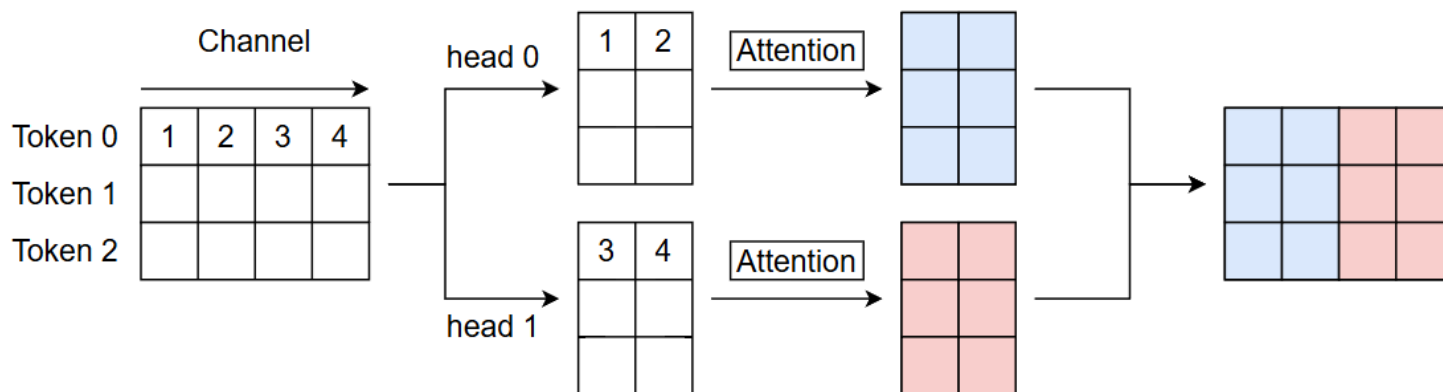
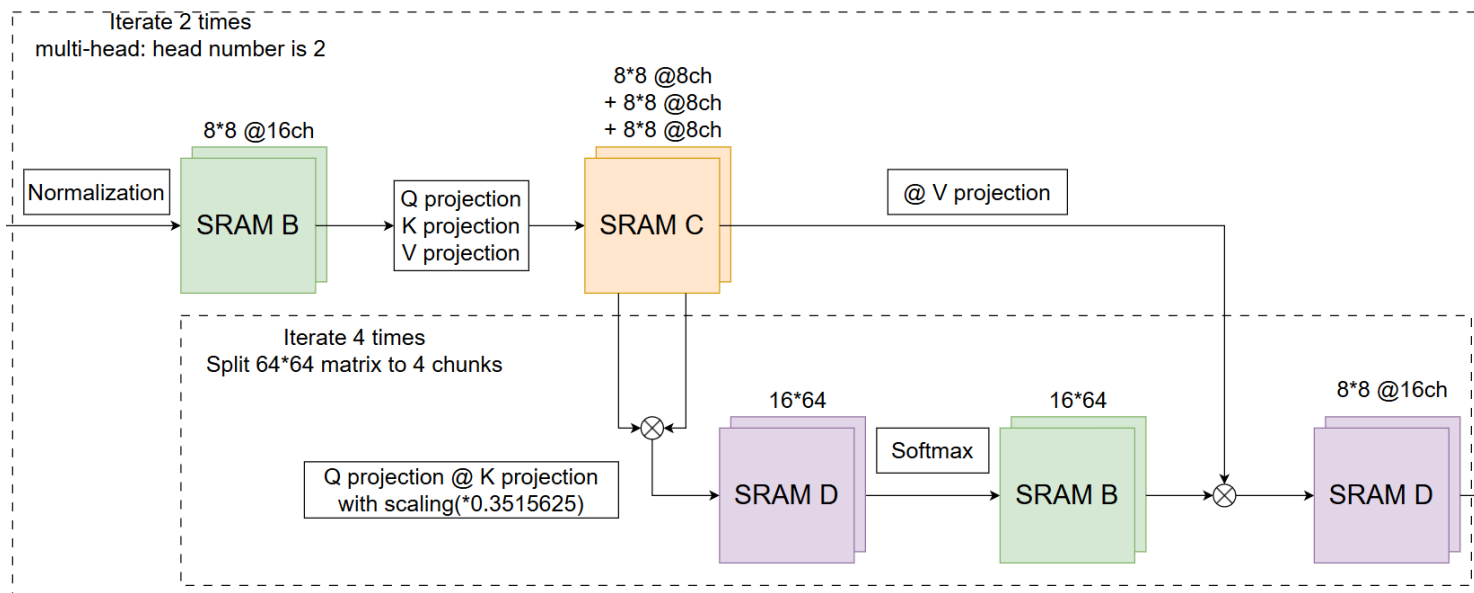


Hardware Implementation

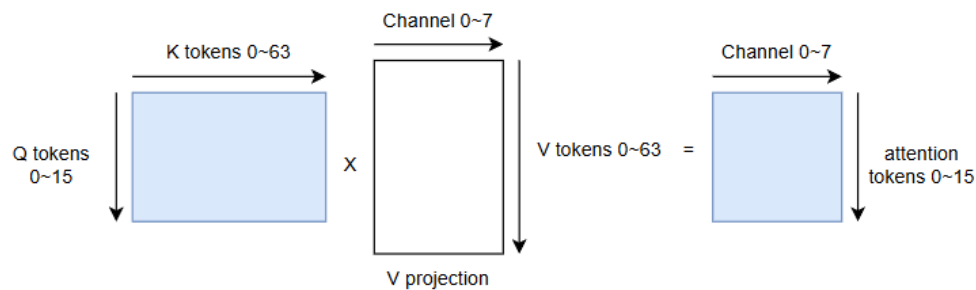
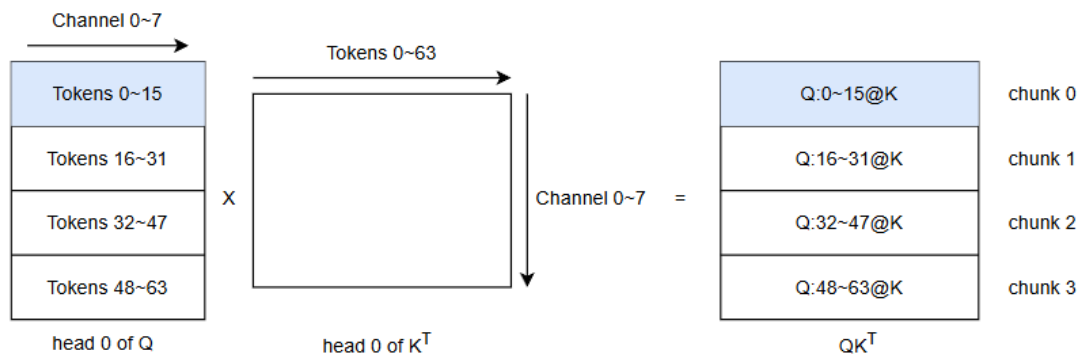
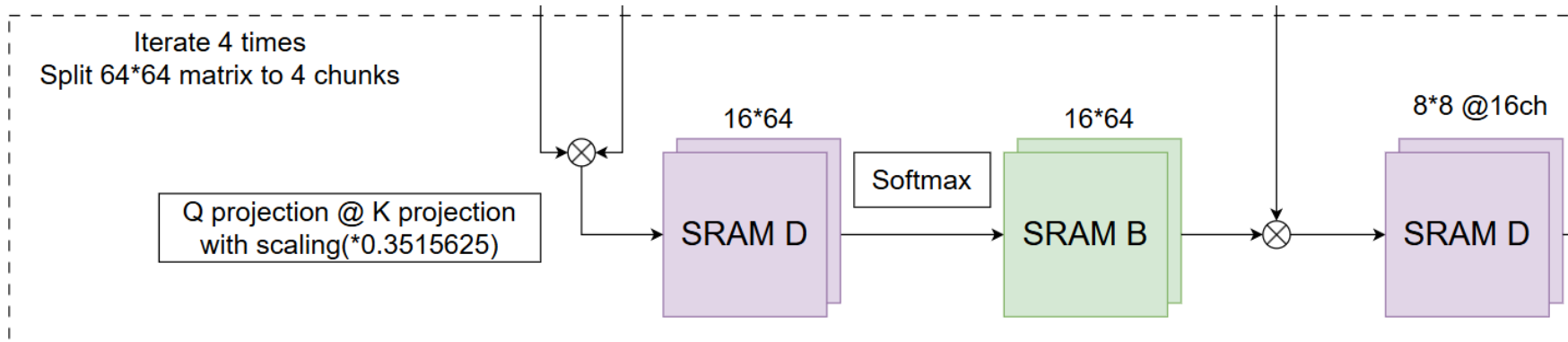
Dataflow



Dataflow

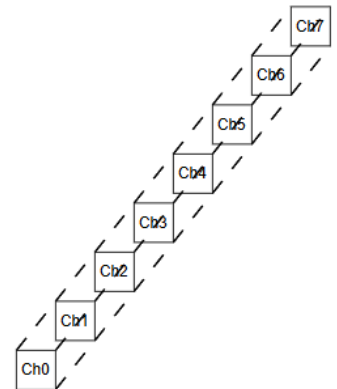
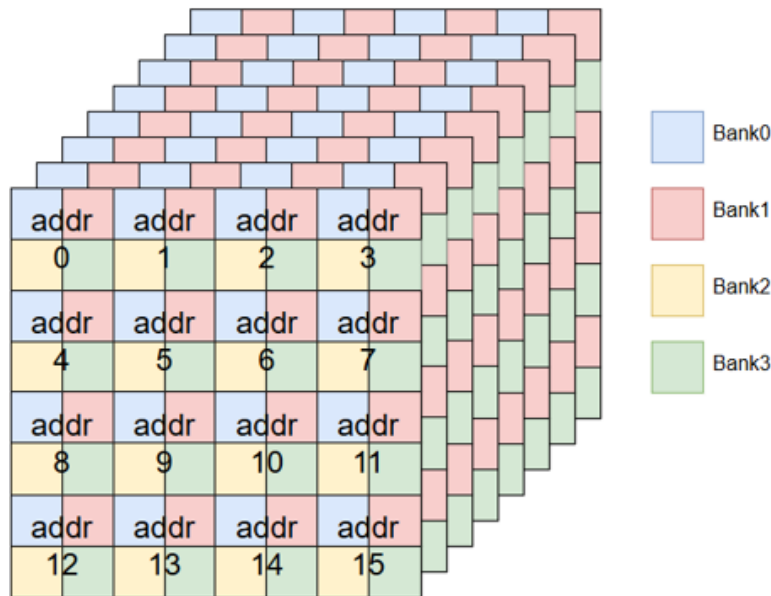


Dataflow



SRAM Group

- SRAM group
 - 4 bank
 - Each bank has an independent R/W channel
 - For one SRAM bank, 80 bit per address – 10 (bit) * 8(channel) = 80 bit
- SRAM size
 - SRAM A : address 0~31
 - SRAM B: address 0~31
 - SRAM C: address 0~47
 - SRAM D: address 0~63



SRAM Mapping

- input, normalization layer, O projection and residual add layer
- 64 patches @16 channels

0	1	4	5	8	9	12	13
2	3	6	7	10	11	14	15
						60	61
						62	63

Token Index

addr	addr	addr	addr				
0	1	2	3				
addr	addr	addr	addr				
4	5	6	7				
addr	addr	addr	addr				
8	9	10	11				
addr	addr	addr	addr				
12	13	14	15				

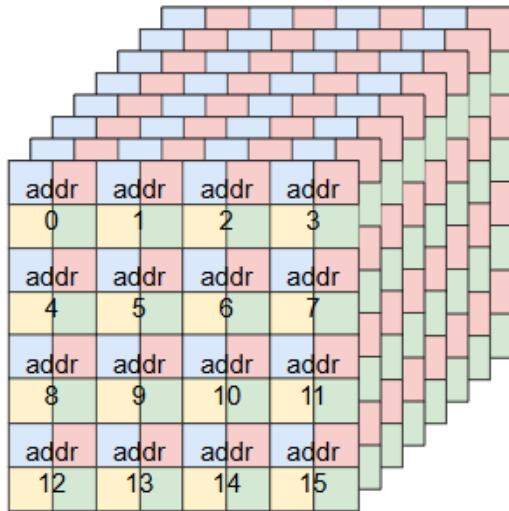
Channel 0~7

addr	addr	addr	addr				
16	17	18	19				
addr	addr	addr	addr				
20	21	22	23				
addr	addr	addr	addr				
24	25	26	27				
addr	addr	addr	addr				
28	29	30	31				

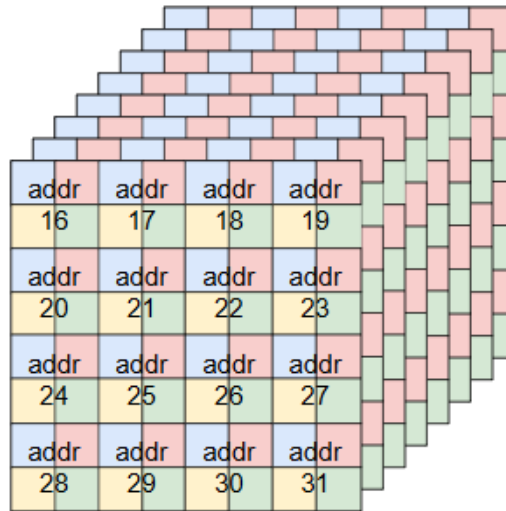
Channel 8~15

SRAM Mapping

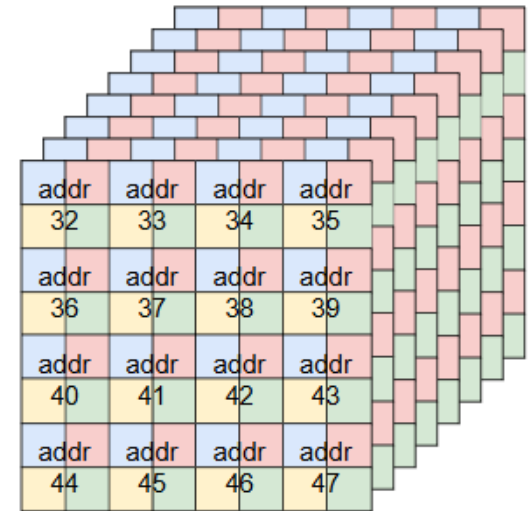
- QKV projection
- (64 patches @8 channels) * 3



Q projection



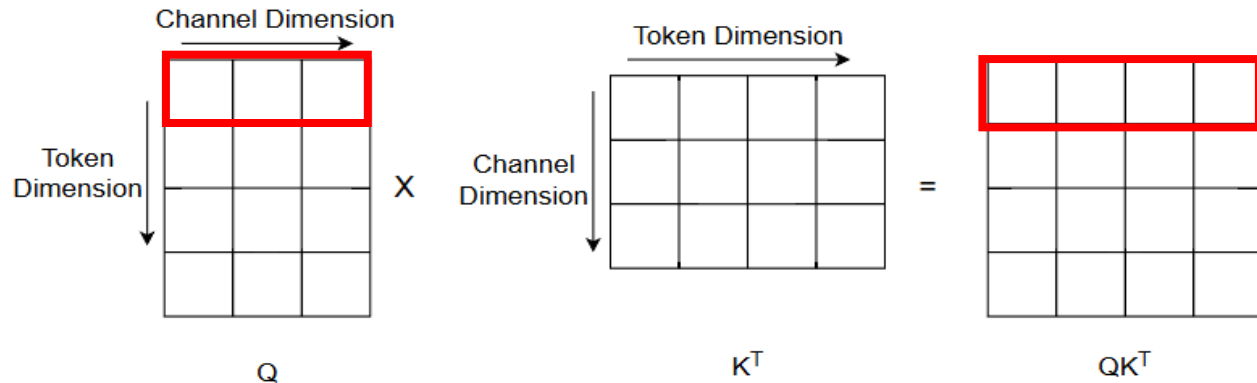
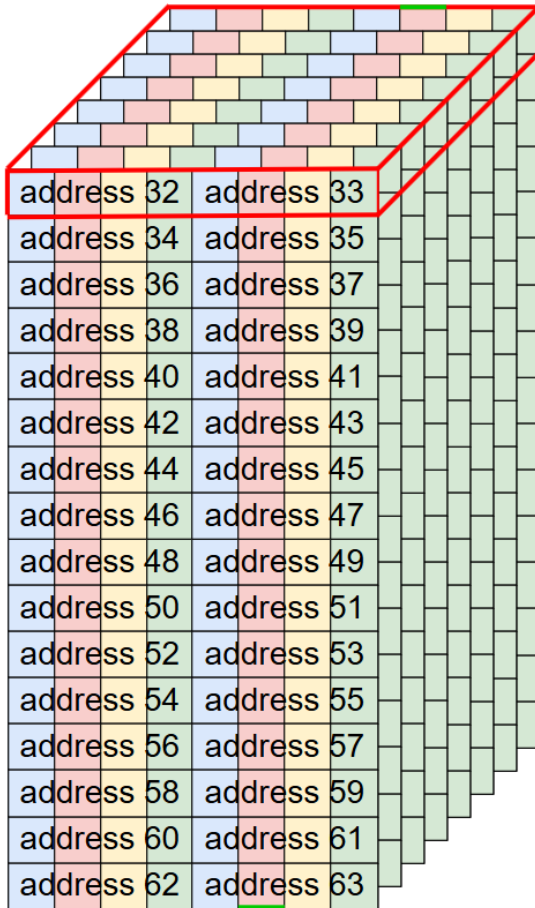
K projection



V projection

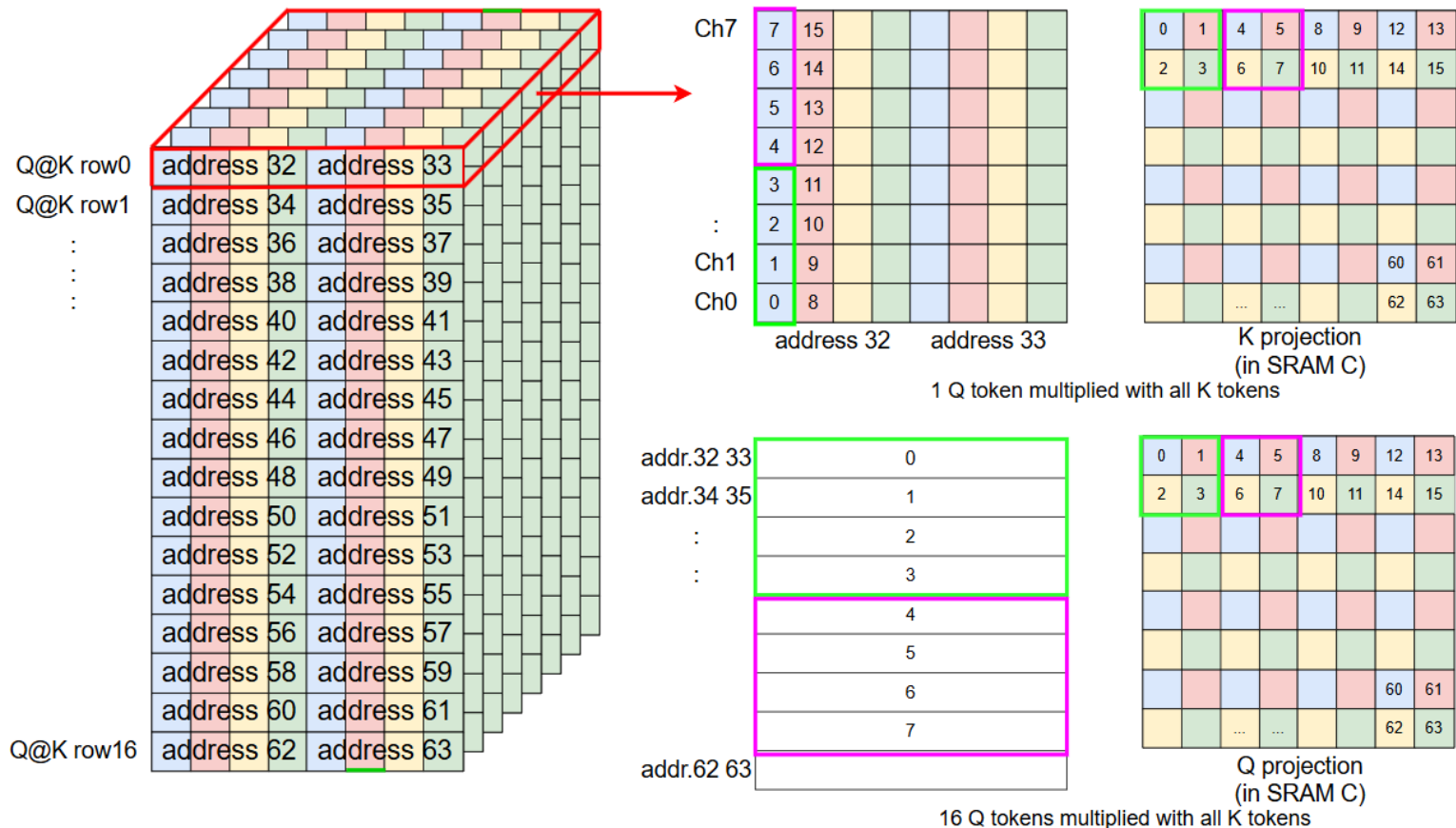
SRAM Mapping

- $Q @ K^T$ and Softmax layer



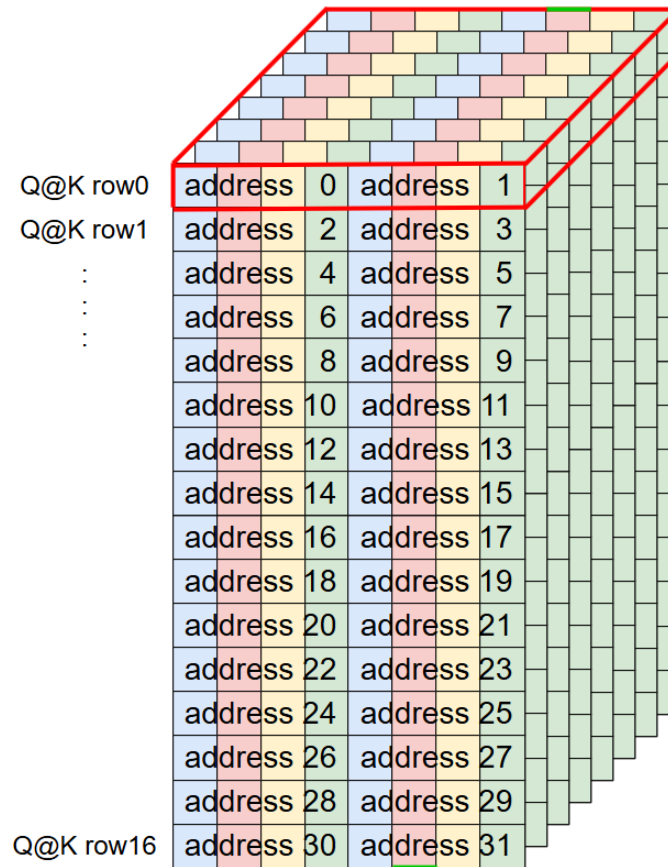
SRAM Mapping

- $Q@K^T$ layer



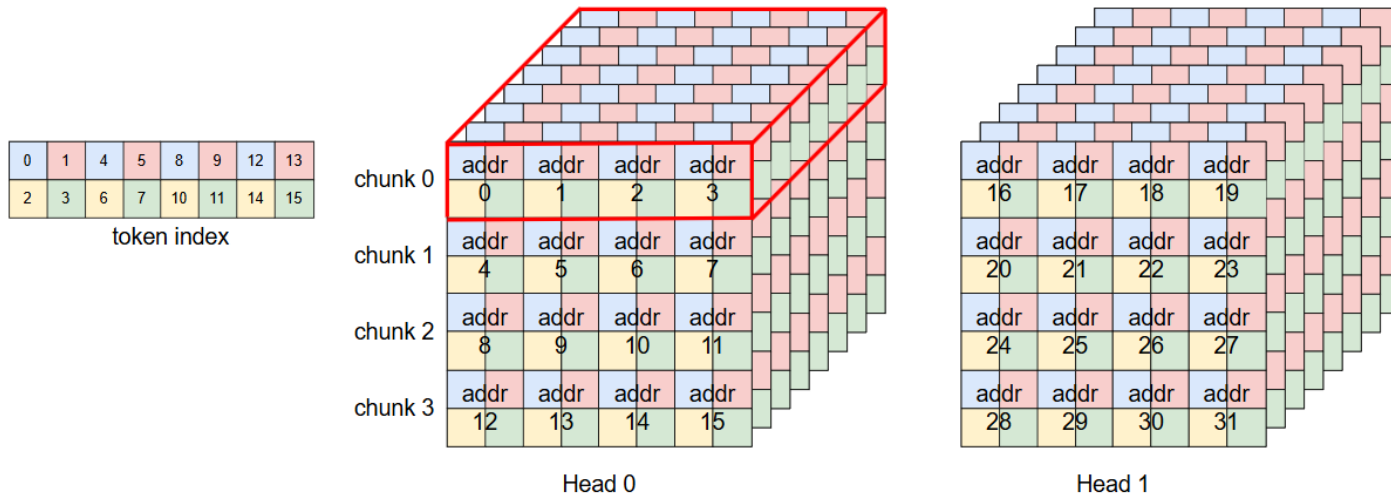
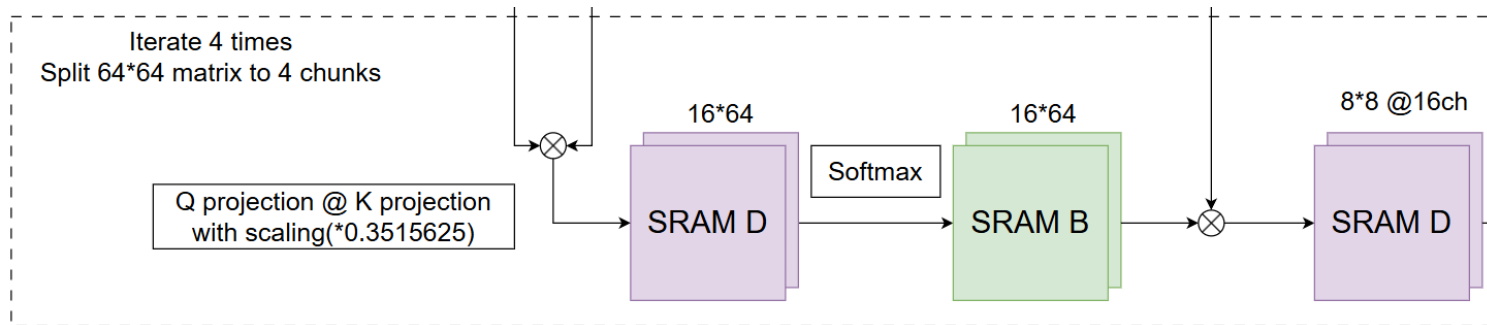
SRAM Mapping

- Softmax layer



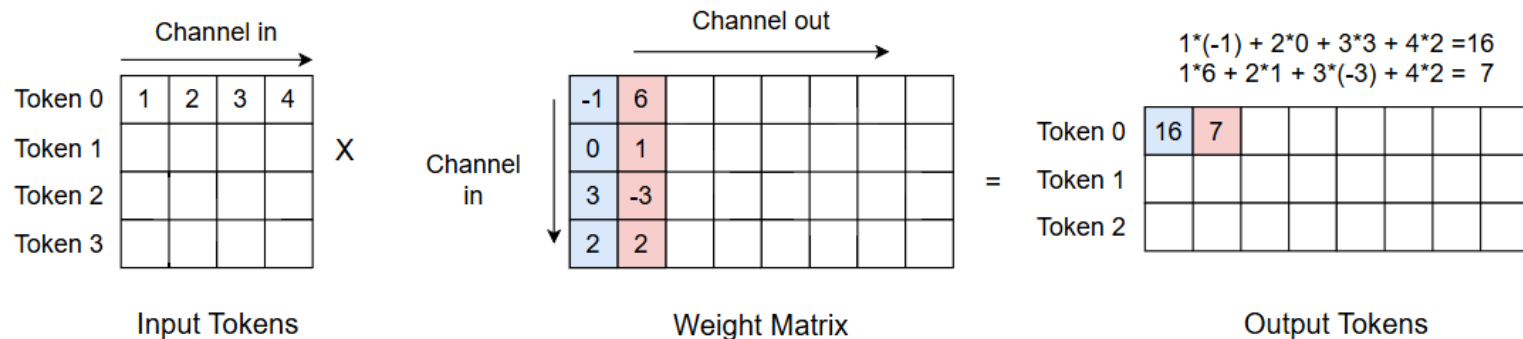
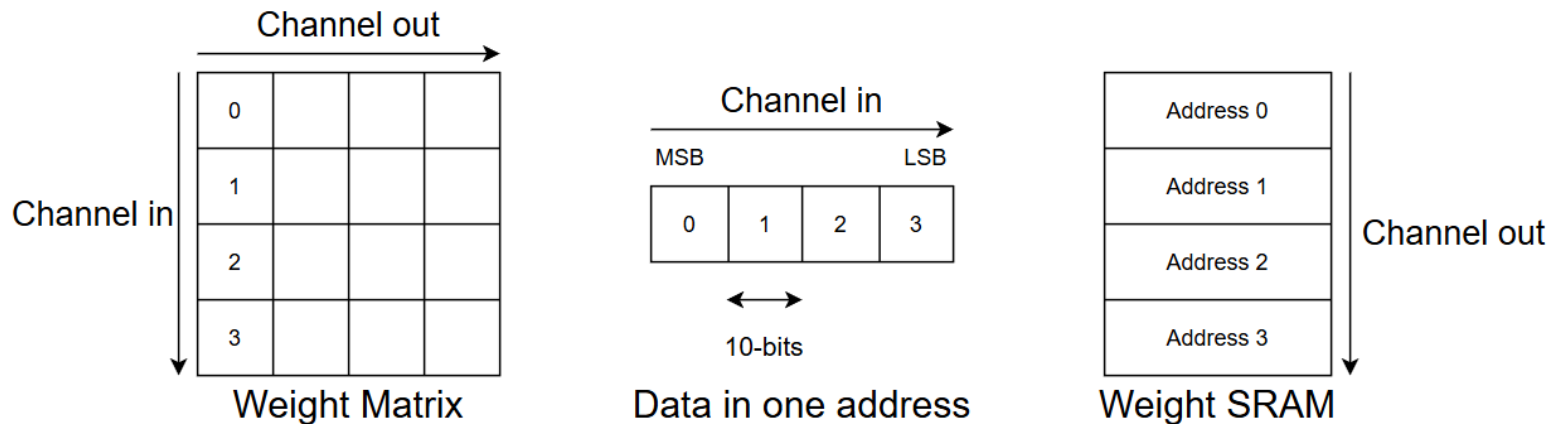
SRAM Mapping

- Attention output layer



SRAM Mapping

- Weight & bias
 - 10 bits
 - Weight: 16 weight per address; Bias: 1 bias per address



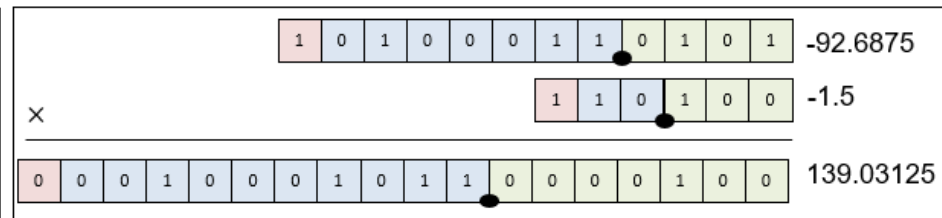
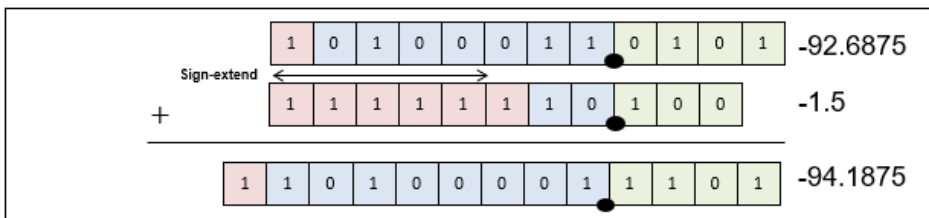
SRAM Mapping

- Weight & bias
 - 10 bits
 - Weight: 16 weight per address; Bias: 1 bias per address

Weight SRAM		Bais SRAM	
ADDR	DATA	ADDR	DATA
0	norm wieght: ({ch0,ch1...ch15})	0	norm bias for ch0
1	Q projection matrix col 0	1	norm bias for ch1
2	Q projection matrix col 1	⋮	norm bias for ch2~14
⋮	Q projection matrix col 2~15	15	norm bias for ch15
17	K projection matrix col 0		
⋮	K projection matrix col 1~15		
33	V projection matrix col 0		
⋮	V projection matrix col 1~15		
49	O projection matrix col 0		
⋮	O projection matrix col 1~14		
64	O projection matrix col 15		

Fix-point number

- Fix point vs. floating point
 - Reduce the MAC area and power overhead
 - Acceptable performance drop
 - Predetermined fraction length(FL)
 - Addition: align the decimal point, $FL_{out} = \max(FL_A, FL_B)$
 - Multiplication: $FL_{out} = FL_A + FL_B$
 - Activation FL = 6 for all layer
- Feature maps: 10 bits, FL = 6
- Parameter & Bias – 10 bits, FL = 7



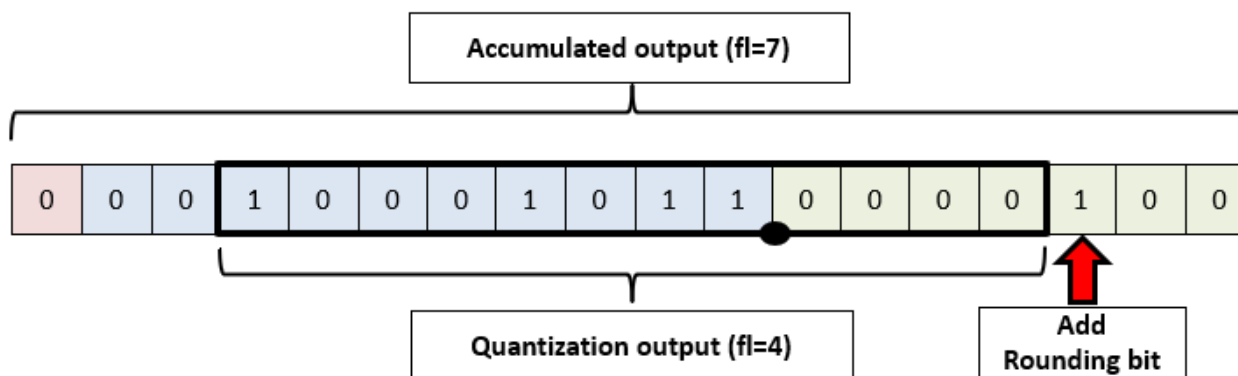
Quantization

- Fix point quantization

- Rounding
- Step
- Clip

```
accumulated_output = convolved_output + bias << (activation_fl + weight_fl - bias_fl);  
rounding_output = accumulated_output + 1 << (weight_fl - 1);  
quantized_output = rounding_output >>> weight_fl  
if (rounding_output > 2047)  
    rounding_output = 2047;  
else if (rounding_output < -2048)  
    rounding_output = -2048;  
else  
    rounding_output = rounding_output [11:0];
```

Add bias
Add rounding bit
Remove extra fraction length
Clip to maximum and minimum



Quantization

- Division
 - Change to positive
 - No need to add rounding bit

$$\begin{array}{r} 101 \\ \overline{) 11010} \\ \underline{(-) 101} \\ 11 \\ \underline{(-) 00} \\ 110 \\ \underline{(-) 101} \\ 1 \end{array}$$

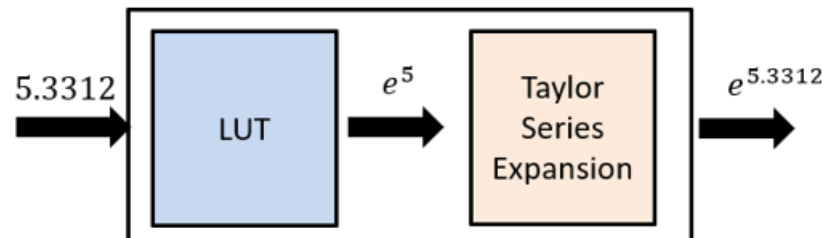
Softmax

- Perform softmax with any method
- Reference LUT table is given (100 bits, fl=50)
- Input (10 bits, fl=6) / Output (10 bits, fl=6)

$$y_i = \frac{e^{x_i}}{\sum_{n=1}^K e^{x_n}} \quad \text{for } i = 1, \dots, K$$

$$f(x) = f(a) + \frac{f^{(1)}(a)}{1!} (x - a) + \frac{f^{(2)}(a)}{2!} (x - a)^2 + \frac{f^{(3)}(a)}{3!} (x - a)^3 + \dots$$

$$e^{10.32} = e^{10} + \frac{e^{10}}{1!} (10.32 - 10) + \frac{e^{10}}{2!} (10.32 - 10)^2 + \frac{e^{10}}{3!} (10.32 - 10)^3 + \dots$$



Grading

- RTL coding and pre-simulation(60%)
 - Part 1: QKV head0 linear projection - 20%
 - Part 2: SoftMax layer (head0 chunk0 only) - 20%
 - Part 3: Multi-head self-attention blocks - 15%
 - Part 4: Residual add - 5%
- Synthesizability check and logic equivalence check(5%)
 - Need to finish Part 1
- PI – ATC(35%)
 - C: cycle counts(100 patterns)

$$\text{Grade point:} \begin{cases} 0\% & \text{if } PI < \text{Threshold} \\ 35 * (\mu + \sigma * \Phi^{-1} * (1 - \frac{\text{rank} - 0.5}{N})) & \text{if } PI \geq \text{Threshold} \end{cases}$$

Note

- For debugging, we have provided display.py for you to check all layers' output feature maps.
- You should NOT use the improper method to finish the homework. We have the right to deduct your score.
 - Try to copy and paste the golden pattern to your code
 - Make the synthesis circuit and RTL code mismatch deliberately
- You can design your own dataflow
 - But!!! Make sure the final result is stored back in the correct SRAM in correct order.
 - For example, if you only complete to part3, SRAM D address0~31 should store correct result.